

```
1  """prelude.py -- Haskell-style Prelude in pure Python, in pedagogical order.
2
3  Pascal discipline: strict define-before-use. Read top to bottom and every name is already defined
4  when you meet it. Sections build on each other:
5
6      1 combinators          2 pairs          3 arithmetic
7      4 list basics          5 higher-order lists  6 folds
8      7 scans                8 streams (lazy)  9 slicing & spans
9      10 sorting & searching 11 strings        12 containers
10     13 nodes                14 grids & windows  15 control
11
12  Within a section names run alphabetically, except that a definition is hoisted ahead of the first
13  name that uses it (foldl before compose, span before break_), so the discipline holds inside
14  sections too.
15
16  Import qualified (import prelude as P): Haskell names shadow builtins by design. Beyond the Prelude
17  proper: the working parts of Data.List, Data.Map, Data.Maybe, Data.Function and friends are folded
18  into the same sections -- grouped by what they do, not by the module they came from. None is
19  Nothing throughout. foldr = foldl(flip(f), reversed(xs), base): iterative, stack-safe, valid only
20  because Python is strict.
21  """
22
23  # ===== 1. combinators ===== (functions about functions)
24
25  const      = lambda x: lambda _: x                # always returns x, ignoring the second argument
26  # curry: converts a two-argument function into a chain of two one-argument functions, so instead
27  # of calling f(x, y), you call curry(f)(x)(y).
28  curry      = lambda f: lambda x: lambda y: f(x, y)
29  flip       = lambda f: (lambda x, y: f(y, x))      # flip f x y = f y x
30  NOTHING    = object()                            # Maybe's Nothing: "no arg given"; test with 'is'
31                                                    # hoisted ahead of foldl, which needs it as a sentinel
32  el
33
34  def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools as reduce
35      it = iter(xs)
36      if base is NOTHING:
37          try:
38              acc = next(it)
39          except StopIteration:
40              raise TypeError("fold of empty sequence with no initial value")
41      else:
42          acc = base
43      for x in it:
44          acc = f(acc, x)
45      return acc
46
47  foldr      = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right fold, via foldl
48  compose    = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id: RIGHTMOST first
49  fromMaybe = lambda d, x: d if x is None else x    # Data.Maybe: default for every None-returning tool
50  id         = lambda x: x                          # shadows builtin id() by design
51  isJust     = lambda x: x is not None              # Data.Maybe: None test as a handable predicate
52  isNothing  = lambda x: x is None                  # Data.Maybe: true when x is Nothing (None)
53  on         = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp 'on' fst
54  partial    = lambda f, *bound: lambda *args: f(*bound, *args) # fix leading args of f, wait for the rest
55  uncurry    = lambda f: lambda p: f(*p)           # inverse of curry: call f with a pair, not two args
56
57  # ===== 2. pairs =====
58
59  fst = lambda p: p[0] # first element of a pair
60  snd = lambda p: p[1] # second element of a pair
61  swap = compose(tuple, reversed) # Data.Tuple
62
63  # ===== 3. arithmetic & logic =====
64
65  div      = lambda a, b: a // b # floors, like Haskell div
66  even     = lambda n: n % 2 == 0 # true for even numbers
67  gcd      = lambda a, b: abs(a) if b == 0 else gcd(b, a % b) # >= 0 like Haskell; depth <= 90
68  halve    = lambda n: n // 2 # e.g. iterate(halve, 1024) -> 1024, 512, 256, ...
69  hypot    = lambda x, y: (x*x + y*y) ** 0.5 # straight-line distance
70
71  def isqrt(n): # floor sqrt without floats (Newton)
72      if n < 0:
73          raise ValueError("isqrt of negative")
74      x = n
75      y = halve(x + 1)
76      while y < x:
77          x, y = y, halve(y + n // y)
78      return x if n else 0
79
80  lcm      = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 # >= 0 like Haskell
81  mod      = lambda a, b: a % b # remainder, sign follows b
82  not_     = lambda b: not b # 'not' is a Python keyword
83  odd      = lambda n: n % 2 == 1 # true for odd numbers
84  otherwise = True # guard fallback: always true
85  pred     = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars
86  quot     = lambda a, b: -(a // b) if (a < 0) != (b < 0) else a // b # truncates, like Haskell quot
87  rem      = lambda a, b: a - b * quot(a, b) # remainder, sign follows a
88  quotRem  = lambda a, b: (quot(a, b), rem(a, b)) # (quot, rem) in one call
```

```

88 signum      = lambda x: (x > 0) - (x < 0)                # -1, 0, or 1 by the sign of x
89 succ        = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1 # next: increments, advances chars
90
91 # ===== 4. list basics =====
92
93 drop         = lambda n, xs: list(xs)[max(n, 0):]        # finite lists; n<0 drops none
94 elem         = lambda x, xs: x in xs                    # true if x is in xs
95 zip_         = lambda a, b: list(zip(a, b))              # shortest wins, any iterable
96 enum         = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) # pairs each x with its index
97 head         = lambda xs: xs[0]                        # first element
98 init         = lambda xs: xs[:-1]                      # all but the last element
99 isPrefixOf   = lambda p, xs: list(xs[:len(p)]) == list(p) # Data.List; strings or lists
100 isSuffixOf   = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not [-len(p):]: [-0:] is ALL
101 last         = lambda xs: xs[-1]                      # last element
102 lookup       = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
103 notElem      = lambda x, xs: x not in xs                # true if x is not in xs
104 nub          = lambda xs: list(dict.fromkeys(xs))        # unique; first seen wins
105 null         = lambda xs: len(xs) == 0                  # true for an empty list
106 pairwise     = lambda xs: list(zip(xs, xs[1:]))         # zip xs (tail xs)
107 replicate    = lambda n, x: [x] * n                    # x repeated n times
108 splitAt      = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0 splits at the front
109 stripPrefix  = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None # Just the rest, or Nothing
110 tail         = lambda xs: xs[1:]                      # all but the first element
111 def transpose(xss):
112     # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
113     xss = [list(xs) for xs in xss] # any iterables in (rows may be one-shot); materialise once
114     n = max(map(len, xss), default=0) # short rows just drop out of later columns (no truncation)
115     return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
116 unzip        = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], []) # inverse of zip: pairs -> two l
117 zip3         = lambda a, b, c: list(zip(a, b, c))        # zip three lists together
118
119 # ===== 5. higher-order lists =====
120
121 filter_      = lambda crit, xs: [x for x in xs if crit(x)] # keep elements where crit holds
122 catMaybes    = partial(filter_, isJust)                  # mapMaybe id
123 concat       = lambda xss: [x for xs in xss for x in xs] # flatten one level of nesting
124 concatMap    = lambda f, xs: [y for x in xs for y in f(x)] # map f over xs, then flatten
125 cross        = lambda a, b: [(x, y) for x in a for y in b] # cartesian; 'product' is the fold
126 # Data.List find: lazy first-match -- folds can't stop early, find can
127 find         = lambda crit, xs: next((x for x in xs if crit(x)), None) # first match, else None
128 map_         = lambda f, xs: [f(x) for x in xs]          # apply f to every element
129 # Data.Maybe mapMaybe: map and drop the Nothings in ONE pass -- the parse-and-filter shape
130 mapMaybe    = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
131 def partition(crit, xs):
132     # (keepers, rest) in ONE pass; crit called once per element
133     yes, no = [], []
134     for x in xs:
135         (yes if crit(x) else no).append(x)
136     return (yes, no)
137 starmap      = lambda f, pairs: [f(*p) for p in pairs]    # map . uncurry
138 zipWith      = lambda f, a, b: [f(x, y) for x, y in zip(a, b)] # combine two lists elementwise
139 zipWith3     = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine three lists elementwise
140
141 # ===== 6. folds ===== (collapse a list to one value)
142 # foldl, foldr and compose are hoisted up to section 1 (combinators): compose needs foldr, foldr needs
143 # foldl, and swap (section 2) needs compose -- so all three must exist before section 2 even begins.
144 foldl1       = lambda f, xs: foldl(f, xs)                # foldl seeded by the first element
145 foldr1       = lambda f, xs: foldl(flip(f), reversed(list(xs))) # foldr seeded by the last element
146 product      = lambda xs: foldl(lambda a, x: a * x, xs, 1) # the numeric fold
147
148 # enumeration folds -- brute-force licenses for small n (say the bound out loud):
149 # Control.Monad replicateM: every length-n word over xs -- cross, n times; |xs|^n
150 replicateM   = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
151 # Data.List subsequences: the powerset; each element DOUBLES acc -- 2^n, fine to n ~ 20
152 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
153
154 # ===== 7. scans ===== (a fold that keeps its history)
155
156 def accumulate(xs, f=None, initial=NOTHING): # scanl / scanl1
157     if f is None:
158         f = lambda a, b: a + b
159     it = iter(xs)
160     if initial is NOTHING:
161         try:
162             acc = next(it)
163         except StopIteration:
164             return
165     else:
166         acc = initial
167     yield acc
168     for x in it:
169         acc = f(acc, x)
170         yield acc
171
172 scanl        = lambda f, z, xs: list(accumulate(xs, f, initial=z)) # foldl, keeping every accumulator
173 scanl1       = lambda f, xs: list(accumulate(xs, f))                # scanl with no explicit base
174 scanr        = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs))))) # foldr, keeping ev

```

```
173  ery acc
174  scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs))))) # scanr with no bas
175  e
176  # ===== 8. streams ===== (lazy, possibly infinite)
177
178  chain      = lambda *iterables: (x for it in iterables for x in it) # concat, lazily
179
180  def count(start=0, step=1): # [start, start+step ..]
181      n = start
182      while True:
183          yield n
184          n += step
185
186  def cycle(xs): # repeat xs forever
187      xs = list(xs)
188      if not xs:
189          raise ValueError("cycle: empty list") # Haskell errors too; the alternative is a silent hang
190      while True:
191          yield from xs
192
193  def iterate(f, x): # iterate f x = [x, f x, f (f x), ..]
194      while True:
195          yield x
196          x = f(x)
197
198  def repeat(x, n=None): # yield x forever, or n times if given
199      if n is None:
200          while True:
201              yield x
202      else:
203          for _ in range(n):
204              yield x
205
206  def unfoldr(f, seed): # Data.List unfoldr: f(seed) -> None | (value, seed')
207      out = [] # iterate's finite twin: grows a list until f says stop --
208      step = f(seed) # no more threading (state, acc) tuples through until
209      while step is not None:
210          val, seed = step
211          out.append(val)
212          step = f(seed)
213      return out
214
215  # ===== 9. slicing & spans ===== (finite views of streams)
216
217  def span(p, xs): # split at first failure, ONE pass; safe on one-shot iters
218      xs = list(xs)
219      i = next((i for i, x in enumerate(xs) if not p(x)), len(xs))
220      return (xs[:i], xs[i:])
221  break_ = lambda p, xs: span(lambda x: not p(x), xs) # split at first hit (opposite of span)
222  chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)] # Data.List.Split; short last ok
223
224  def dropwhile(crit, xs): # generator: drop the leading run where crit holds
225      it = iter(xs)
226      for x in it:
227          if not crit(x):
228              yield x
229              break
230      yield from it
231
232  dropWhile = lambda p, xs: list(dropwhile(p, xs)) # drop the leading run where p holds, as a list
233  inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)] # Data.List: every prefix, [] first
234
235  def islice(iterable, stop): # take
236      it = iter(iterable)
237      for _ in range(stop):
238          try:
239              yield next(it)
240          except StopIteration:
241              return
242
243  tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)] # every suffix; substrings start here
244  take = lambda n, xs: list(islice(iter(xs), n)) # works on infinite streams
245
246  def takeuntil(crit, xs): # like takewhile(not . crit), but INCLUDES the first hit
247      for x in xs:
248          yield x
249          if crit(x):
250              return
251
252  def takewhile(crit, xs): # generator: yield the leading run where crit holds
253      for x in xs:
254          if not crit(x):
255              return
256          yield x
257
258  takeWhile = lambda p, xs: list(takewhile(p, xs)) # take the leading run where p holds, as a list
```

```
259
260 # ===== 10. sorting & searching =====
261
262 def bisect_left(a, x):
263     lo, hi = 0, len(a)
264     while lo < hi:
265         mid = halve(lo + hi)
266         if a[mid] < x:
267             lo = mid + 1
268         else:
269             hi = mid
270     return lo
271
272 def bisect_right(a, x):
273     lo, hi = 0, len(a)
274     while lo < hi:
275         mid = halve(lo + hi)
276         if a[mid] <= x:
277             lo = mid + 1
278         else:
279             hi = mid
280     return lo
281
282 def cmp_to_key(cmp):
283     class K:
284         def __init__(self, x): self.x = x
285         def __lt__(self, other): return cmp(self.x, other.x) < 0
286     return K
287
288 maxOn = lambda f, xs: max(xs, key=f)
289
290 def merge(a, b):
291     out, i, j = [], 0, 0
292     while i < len(a) and j < len(b):
293         if a[i] <= b[j]:
294             out.append(a[i]); i += 1
295         else:
296             out.append(b[j]); j += 1
297     return out + a[i:] + b[j:]
298
299 minOn = lambda f, xs: min(xs, key=f)
300 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))
301 sortOn = lambda f, xs: sorted(xs, key=f)
302
303 # ===== 11. strings =====
304
305 lines = str.splitlines
306 unlines = lambda ls: "".join(l + "\n" for l in ls)
307 unwords = " ".join
308 words = str.split
309
310 # ===== 12. containers =====
311
312 # multiset ops on Counters (use .get, not [] -- indexing a Counter autovivifies zeros):
313 bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)}
314 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k, 0)))}
315 bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())
316 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
317
318 class defaultdict(dict):
319     def __init__(self, factory=None, *args, **kw):
320         super().__init__(*args, **kw)
321         self.factory = factory
322     def __missing__(self, key):
323         if self.factory is None:
324             raise KeyError(key)
325         self[key] = self.factory()
326         return self[key]
327
328 def Counter(xs):
329     d = defaultdict(int)
330     for x in xs:
331         d[x] += 1
332     return d
333
334 class deque:
335     def __init__(self, xs=()):
336         self._in, self._out = [], list(reversed(list(xs)))
337     def append(self, x): self._in.append(x)
338     def appendleft(self, x): self._out.append(x)
339     def pop(self):
340         if not self._in:
341             self._in, self._out = list(reversed(self._out)), []
342         return self._in.pop()
343     def popleft(self):
344         if not self._out:
345             self._out, self._in = list(reversed(self._in)), []
```

```
346         return self._out.pop()
347     def __len__(self): return len(self._in) + len(self._out)
348     def __iter__(self): return iter(self._out[::-1] + self._in)
349
350 def dsu(n):                                     # union-find, path halving; union -> False if already joined
351     parent = list(range(n))
352     def find(x):
353         while parent[x] != x:
354             parent[x] = parent[parent[x]]
355             x = parent[x]
356         return x
357     def union(a, b):
358         ra, rb = find(a), find(b)
359         if ra == rb:
360             return False
361         parent[ra] = rb
362         return True
363     return find, union
364
365 def fromListWith(f, pairs):                     # Data.Map fromListWith: THE dict-building fold; f(new, old)
366     d = {}                                     # like insertWith -- so grouping with concat REVERSES each
367     for k, v in pairs:                         # group (the classic Haskell gotcha); use add for counts
368         d[k] = f(v, d[k]) if k in d else v
369     return d                                   # Counter == fromListWith(add) over (x, 1);
370                                           # grouping == fromListWith(++) over (k, [v])
371
372 def getpath(t, ks, default=None):              # read along ks WITHOUT autovivifying
373     for k in ks:
374         if not isinstance(t, dict) or k not in t:
375             return default
376         t = t[k]
377     return t
378
379 def groupBy(eq, xs):                           # Data.List groupBy: runs of CONSECUTIVE elements; each new
380     out = []                                  # element is compared via eq against the run's FIRST element,
381     for x in xs:                               # exactly like Haskell (span-based) -- not its neighbor
382         if out and eq(out[-1][0], x):
383             out[-1].append(x)
384         else:
385             out.append([x])
386     return out
387
388 group = lambda xs: groupBy(lambda a, b: a == b, xs) # Data.List group: runs of equals, [[a]] -- no keys
389
390 def heappop(h):                                # min-heap on a plain list: sift-down
391     h[0], h[-1] = h[-1], h[0]
392     top = h.pop()
393     i, n = 0, len(h)
394     while True:
395         s, l, r = i, 2*i + 1, 2*i + 2
396         if l < n and h[l] < h[s]: s = l
397         if r < n and h[r] < h[s]: s = r
398         if s == i:
399             return top
400         h[i], h[s] = h[s], h[i]
401         i = s
402
403 def heappush(h, x):                            # min-heap on a plain list: sift-up
404     h.append(x)
405     i = len(h) - 1
406     while i and h[halve(i - 1)] > h[i]:
407         h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
408         i = halve(i - 1)
409
410 insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v} # Data.Map insertWith: PERSISTENT
411                                           # (fresh dict); f(new, old)
412 ITree = lambda: defaultdict(ITree)            # infinitely-nestable defaultdict: a tree that grows on access
413
414 def paths(t):                                  # cata for Tree/ITree: [(keypath, leaf)]
415     # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit
416     if not isinstance(t, dict) or not t:      # leaf = non-dict value, or empty node
417         return [([], t)]
418     return [(k + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]
419
420 leaves = lambda t: [l for _, l in paths(t)]    # every leaf value in a Tree/ITree, in path order
421
422 def setpath(t, ks, v):                        # write leaf v at key path ks (autovivifies interior)
423     for k in ks[:-1]:
424         t = t[k]
425     t[ks[-1]] = v                             # NB: clobbers any subtree already at ks
426
427 Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1          # fixed-depth nested defaultdict;
428                             else defaultdict(lambda: Tree(depth - 1, leaf))) # leaves default to leaf()
429
430 unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
431                                           # Data.Map unionWith: f(a's value, b's value) on shared keys;
432                                           # bag_union == unionWith(max), merge-sum == unionWith(add)
433
```

```
434 # ===== 13. nodes ===== (the LeetCode givens: binary trees & linked lists)
435
436 class ListNode:                                # the LeetCode linked list
437     def __init__(self, val=0, nxt=None):
438         self.val, self.next = val, nxt
439
440 class TreeNode:                                # the LeetCode binary tree
441     def __init__(self, val=0, left=None, right=None):
442         self.val, self.left, self.right = val, left, right
443
444 from_list = lambda xs: foldr(ListNode, xs, None) # build; foldr of ListNode
445
446 def has_cycle(node):                            # Floyd: fast laps slow iff a cycle exists
447     slow = fast = node
448     while fast and fast.next:
449         slow, fast = slow.next, fast.next.next
450         if slow is fast:
451             return True
452     return False
453
454 # tfold: the tree cata -- f(val, l_acc, r_acc), z for the empty tree; recursion depth = tree height
455 tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
456 inorder = lambda t: tfold(lambda v, l, r: l + [v] + r, [], t) # left, root, right
457
458 def levelorder(t):                              # BFS: the deque earning its keep
459     out, q = [], deque([t] if t else [])
460     while len(q):
461         n = q.popleft()
462         out.append(n.val)
463         if n.left: q.append(n.left)
464         if n.right: q.append(n.right)
465     return out
466
467 def merge_lists(a, b):                          # merge two sorted lists; dummy-head idiom
468     dummy = tail_ = ListNode()
469     while a and b:
470         if a.val <= b.val:
471             tail_.next, a = a, a.next
472         else:
473             tail_.next, b = b, b.next
474         tail_ = tail_.next
475     tail_.next = a or b
476     return dummy.next
477
478 def middle(node):                               # fast/slow: second middle for even lengths
479     slow = fast = node
480     while fast and fast.next:
481         slow, fast = slow.next, fast.next.next
482     return slow
483
484 postorder = lambda t: tfold(lambda v, l, r: l + r + [v], [], t) # left, right, root
485 preorder = lambda t: tfold(lambda v, l, r: [v] + l + r, [], t) # root, left, right
486
487 def reverse_list(node):                         # three-pointer; prev is a fold accumulator
488     prev = None
489     while node:
490         node.next, prev, node = prev, node, node.next
491     return prev
492
493 tdepth = lambda t: tfold(lambda _, l, r: 1 + max(l, r), 0, t) # height of the tree
494 to_list = lambda node: list(unfoldr(lambda n: None if n is None else (n.val, n.next), node)) # ListNode
495 chain -> list
496 tsize = lambda t: tfold(lambda _, l, r: 1 + l + r, 0, t) # number of nodes in the tree
497
498 # ===== 14. grids & windows =====
499
500 def longest_window(xs, valid, add, shed):        # sliding-window skeleton; state lives in the closures
501     lo = best = 0
502     for hi in range(len(xs)):
503         add(xs[hi])
504         while not valid():
505             shed(xs[lo])
506             lo += 1
507         best = max(best, hi - lo + 1)
508     return best
509
510 def neighbors4(r, c, R, C):                     # in-bounds orthogonal neighbors
511     for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
512         if 0 <= r + dr < R and 0 <= c + dc < C:
513             yield r + dr, c + dc
514
515 def neighbors8(r, c, R, C):                     # ... plus diagonals
516     for dr in (-1, 0, 1):
517         for dc in (-1, 0, 1):
518             if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
519                 yield r + dr, c + dc
520
521 # ===== 15. control =====
```

```
521
522 def memo(f):                                     # unbounded memoizer; enough for DP (no eviction by design)
523     cache = {}
524     def wrapped(*args, **kw):
525         key = (args, frozenset(kw.items())) if kw else args
526         if key not in cache:
527             cache[key] = f(*args, **kw)
528         return cache[key]
529     wrapped.cache = cache                         # peek at the DP table if curious
530     return wrapped
531
532 def until(p, f, x):                               # until p f x ? loop, not recursion (unbounded depth)
533     while not p(x):
534         x = f(x)
535     return x
end
```